Lundi 19/10 — Fonctions et démarche de débogage

Kevin Bardoux

lundi 19 octobre 2026

Table des matières

Objectifs 2

Matin — Fonctions & portée · démarche de résolution de problème (lire l'erreur, isoler, tester) — exercices 2

Rappel (5 min) 2

Récupérer les fichiers (5 min) 2

Apport 1 — Les fonctions (10 min) 3

Fonctions — attention et vérification 6

Exercices l1 à l3 — Premières fonctions (30 min) 6

Apport 2 — La portée (10 min) 7

Portée — attention et vérification 9

Apport 3 — Déboguer avec méthode (20 min) 9

Débogage — attention et vérification 10

Débogage — l4 et l5 (30 min) 10

Si le temps le permet — l6 (dernière lettre) 11

À retenir — ce matin 11

Débrief 12

Après-midi — TP — « Problem set 1 » (validé par les tests) 12

Apport 4 — Le reste `%` (10 min) 12

Reste — attention et vérification 13

Consignes (1 h 50) 13

Les quatre fonctions 13

Anglais technique — *Functions* (MDN) 14

Si le temps le permet — p1-premier, puis p1-table 14

Pour aller plus loin 14

À retenir — cet après-midi 15
Sources 15

> Note
>
> **Compétences visées** : résolution de problème · apprendre en continu (anglais) **Livrable** : les quatre premières fonctions du Problem set 1 au vert (`p1-premier` si le temps le permet).

Objectifs

- Écrire une fonction avec des paramètres et un `return`.
- Expliquer pourquoi une variable d'une fonction n'existe pas dehors.
- Déboguer en quatre temps : lire, isoler, tester, corriger.
- Faire passer les tests du Problem set 1.

Comment se passe la journée. Le matin, trois notions : la fonction, la portée, la démarche de débogage. L'après-midi, une : le reste `%`. Chacune suit le même ordre : pourquoi, définition, exemple, pas à pas, erreur fréquente, vérification ; un encadré « À retenir » termine chaque demi-journée. L'après-midi, le Problem set 1, validé par les tests.

Matin — Fonctions & portée · démarche de résolution de problème (lire l'erreur, isoler, tester) — exercices

Rappel (5 min)

1. Que fait `npx eslint .` ?
2. `while` ou `for` : lequel pour compter de 1 à 10 ?
3. Que vaut `01001` avec les cartes 16, 8, 4, 2, 1 ?

Trois questions courtes, sans regarder vos notes : la veille, la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

Récupérer les fichiers (5 min)

1. Téléchargez l'archive de la semaine.

2. Extrayez-la ; renommez le dossier `semaine-03` ; placez-le dans `dwwm`.
3. Terminal : `cd ~/dwwm/semaine-03`, puis `npm install`.
4. VS Code : *File*, *Open Folder…*, puis `dwwm/semaine-03`.
5. ECF vendredi : un **aide-mémoire** manuscrit d'une page est autorisé.

Même méthode que la semaine dernière pour extraire l'archive : sous Windows, clic droit, *Extraire tout…* ; sous macOS, double-clic ; sous Linux, clic droit, *Extraire ici*. `npm install` installe ESLint pour cette semaine.

L'ECF de vendredi se passe sans IA et sans internet. Un seul document est autorisé : un **aide-mémoire papier d'une page A4, recto, écrit à la main par vous**. Commencez-le dès aujourd'hui : notez-y ce que vous cherchez plusieurs fois cette semaine (la forme d'une fonction, une boucle `for`, `length - 1`…), avec des exemples courts plutôt que des définitions. Ce sera la règle de tous les ECF de la formation.

Vous devez voir, après l'étape 3 :

```
PS C:\Users\stagiaire> cd ~/dwwm/semaine-03
PS C:\Users\stagiaire\dwwm\semaine-03> npm install

added 79 packages, and audited 80 packages in 8s

18 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Le nombre de paquets et la durée peuvent changer : l'installation est réussie si aucune ligne ne commence par `npm error`.

Apport 1 — Les fonctions (10 min)

- **Pourquoi** : écrire un calcul une fois, l'utiliser partout.
- **Fonction** : un nom, des paramètres, un résultat renvoyé.

Pourquoi. En semaine 2, chaque programme lisait ses valeurs dans `process.argv` et affichait un résultat. Pour réutiliser un calcul (une aire, un prix TTC) avec d'autres valeurs, et pour le tester, on l'enferme dans une fonction.

Définition. Une fonction est un ensemble d'instructions qui effectue une tâche ou calcule une valeur ; on la déclare avec le mot-clé `function`, un nom, des **paramètres** entre parenthèses et des instructions entre accolades (MDN Web Docs, s. d.-b). L'instruction `return` termine la fonction et indique la valeur renvoyée (MDN Web Docs, s. d.-i). En semaine 2, le bloc personnalisé Scratch (rubrique « Pour aller plus loin ») était déjà une fonction.

Exemple, commenté ligne par ligne.

```
function square(n) {
  return n * n;
}

const result = square(4);
console.log(result);
```

Ligne 1

`function` déclare une fonction nommée `square`, avec un paramètre `n` : la valeur qu'elle recevra à chaque appel.

Ligne 2

`return` arrête la fonction et renvoie le résultat de `n * n` à l'endroit qui l'a appelée.

Ligne 3

L'accolade ferme la fonction. Jusqu'ici, rien ne s'exécute : la fonction est seulement déclarée.

Ligne 5

`square(4)` appelle la fonction : pendant cet appel, `n` vaut `4`. La valeur renvoyée, `16`, est rangée dans `result`.

Ligne 6

`console.log` affiche `16` dans le terminal.

Pas à pas. Le formateur tape chaque étape dans un fichier `essai-fonction.js` ; vous la refaites aussitôt.

1. Écrivez la fonction `square` (lignes 1 à 3). Lancez `node essai-fonction.js` : rien ne s'affiche. Déclarer une fonction ne l'exécute pas.
2. Ajoutez la ligne 5 : `square(4)` **appelle** la fonction. Le paramètre `n` vaut `4` pendant cet appel.
3. `return n * n` renvoie `16` : `result` vaut `16`.
4. Ajoutez la ligne 6 et relancez : le terminal affiche `16`.
5. Ouvrez `l1-bonjour.js`, le premier exercice. Chaque exercice de la semaine a cette forme :

```
export function greet(firstName) {
  // TODO: replace the line below with your code.
  return `À faire (${firstName})`;
}
```

6. Remplacez le `return` par le vôtre ; gardez `export`. `export` rend la fonction utilisable depuis un autre fichier, ici le fichier de test (MDN Web Docs, s. d.-a) (les modules sont détaillés en semaine 9). `node --test l1-bonjour.test.js` appelle votre fonction et vérifie ce qu'elle **renvoie**.

Vous devez voir : à l'étape 1, rien ; à l'étape 4, `16`.

```
PS C:\Users\stagiaire\dwwm\semaine-03> node essai-
fonction.js
PS C:\Users\stagiaire\dwwm\semaine-03> node essai-
fonction.js
16
```

À l'étape 6, avant d'écrire votre `return`, le test est rouge. Voici le début et les lignes utiles ; … remplace les lignes à ignorer :

```
PS C:\Users\stagiaire\dwwm\semaine-03> node --test l1-
bonjour.test.js
✖ Ada (5.995392ms)
✖ Linus (0.933632ms)
```

```
ℹ tests 2
ℹ suites 0
ℹ pass 0
ℹ fail 2
…
  + 'À faire (Ada)'
  - 'Bonjour, Ada !'
…
```

`+` montre ce que renvoie votre fonction, `-` ce qu'elle devrait renvoyer : c'est la lecture d'un test rouge de la semaine 2.

Fonctions — attention et vérification

> ⚠ Attention
>
> `console.log` affiche, `return` renvoie. Si le test reçoit `undefined`, la fonction affiche au lieu de renvoyer : remplacez `console.log(…)` par `return …`.

- **Vérifiez** : que renvoie `square(3)` ? Et `square(-2)` ?

Exercices l1 à l3 — Premières fonctions (30 min)

1. Ensemble (5 min) : **l1-bonjour**.
2. Seuls : **l2-aire-rectangle**, puis **l3-majorite**.
3. Pour chacun : ses tests, puis ESLint sur son seul fichier.
4. **Réussi si** : tests au vert, aucune alerte.

- **l1-bonjour** : `greet('Ada')` renvoie `'Bonjour, Ada !'`.
- **l2-aire-rectangle** : largeur × hauteur, `null` si une dimension est négative.
- **l3-majorite** : l'exercice 06 de la semaine 2, écrit en fonction.
- Tests d'un exercice : `node --test l1-bonjour.test.js` (et ainsi de suite).
- ESLint sur l'exercice en cours, comme les tests : `npx eslint l1-bonjour.js`, puis `npx eslint l2-aire-rectangle.js`, puis `npx eslint l3-majorite.js`.

Vous devez voir, quand l1 est fini : `ℹ fail 0` pour les tests ; pour ESLint, rien du tout. Quand le fichier respecte toutes les règles, ESLint n'affiche aucune ligne : l'invite revient directement.

```
PS C:\Users\stagiaire\dwwm\semaine-03> node --test l1-
bonjour.test.js
✓ Ada (9.245182ms)
✓ Linus (0.609778ms)
ℹ tests 2
ℹ suites 0
ℹ pass 2
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 226.42849
PS C:\Users\stagiaire\dwwm\semaine-03> npx eslint l1-
bonjour.js
PS C:\Users\stagiaire\dwwm\semaine-03>
```

Les durées entre parenthèses changent à chaque lancement : elles ne comptent pas.

Apport 2 — La portée (10 min)

- **Pourquoi** : un même nom dans deux fonctions, sans conflit.
- **Portée** : une variable n'existe que dans son bloc `{ … }`.

Pourquoi. Dans un programme, plusieurs fonctions utilisent des variables comme `result` ou `total`. Si elles se voyaient toutes, l'une pourrait modifier celle de l'autre par erreur.

Définition. La portée est la partie du programme où une variable est visible et utilisable (MDN Web Docs, s. d.-f) : une variable créée avec `let` ou `const` dans une fonction, ou dans un bloc `{ … }`, n'existe que **dans** cette fonction ou ce bloc (MDN Web Docs, s. d.-b).

Exemple.

```
function double(n) {
  const result = n * 2;
  return result;
}

console.log(double(3)); // 6
console.log(result);    // ReferenceError
```

Pas à pas. Le formateur tape le code dans `essai-portee.js` ; vous le refaites aussitôt.

1. Écrivez les lignes 1 à 6. Lancez le fichier : il affiche `6`.
2. Ajoutez la ligne 7 et relancez : `ReferenceError: result is not defined`.
3. Lisez l'erreur : `result` n'existe pas à cet endroit, il n'existe que dans `double`.
4. Pour garder le résultat, stockez ce que renvoie la fonction : `const doubled = double(3);`.

Vous devez voir, à l'étape 2 : `6`, puis l'erreur. Les lignes qui commencent par `at` sont des détails pour plus tard : ignorez-les.

```
PS C:\Users\stagiaire\dwwm\semaine-03> node essai-portee.js
6
file:///C:/Users/stagiaire/dwwm/semaine-03/essai-portee.js:7
console.log(result);    // ReferenceError
            ^

ReferenceError: result is not defined
    at file:///C:/Users/stagiaire/dwwm/semaine-03/essai-portee.js:7:13
    at ModuleJob.run (node:internal/modules/esm/module_job:561:25)
    at async node:internal/modules/esm/loader:647:26
    at async asyncRunEntryPointWithESMLoader (node:internal/modules/run_main:101:5)
```

```
Node.js v24.21.0
```

Portée — attention et vérification

> ⚠ Attention
>
> `ReferenceError: … is not defined` : la variable est utilisée hors de son bloc, ou son nom est mal écrit. Vérifiez où elle est déclarée.

- **Vérifiez** : `if (true) { const x = 1; }` puis `x` : que se passe-t-il ?

Apport 3 — Déboguer avec méthode (20 min)

- **Pourquoi** : trouver une erreur sans modifier le code au hasard.
- **Méthode** : lire, isoler, tester, corriger.

Pourquoi. Changer le code au hasard jusqu'à ce que « ça marche » prend du temps et crée de nouvelles erreurs. Une démarche en quatre temps trouve la cause plus vite, et elle se raconte devant un jury.

Définition. Déboguer, c'est trouver et corriger la cause d'un comportement faux, en quatre temps : **lire** l'erreur (son type, son message, la ligne) ; **isoler** l'endroit où le problème apparaît (un `console.log` aide) ; **tester** le cas précis qui échoue (vide, limite, absent) ; **corriger**, puis relancer **tous** les tests.

Exemple. Une erreur affichée par Node.js :

```
/…/err.js:2
  return word[word.length - 1];
                   ^

TypeError: Cannot read properties of undefined (reading
'length')
    at lastLetter (/…/err.js:2:20)
```

Pas à pas. Le formateur lit l'erreur à voix haute ; vous notez chaque réponse.

1. **Lire** le type : `TypeError`, une opération impossible sur une valeur du mauvais type (MDN Web Docs, s. d.-j). (Une `ReferenceError` signale une variable qui n'existe pas (MDN Web Docs, s. d.-g).)
2. **Lire** le message : on lit `length` sur une valeur `undefined`.
3. **Lire** l'endroit : fichier `err.js`, ligne 2, colonne 20, dans la fonction `lastLetter`.
4. **Isoler** : quelle valeur vaut `undefined` ? `word`. La fonction a été appelée sans argument.
5. **Tester** : appelez `lastLetter()` sans argument, puis `lastLetter('chat')`, pour comparer.
6. **Corriger** : ajoutez une garde au début de la fonction, puis relancez tous les tests.

Message reproduit tel que Node.js l'affiche. La fiche Lire un message d'erreur rassemble les autres messages de Node (`ReferenceError`, `SyntaxError`, fichier introuvable).

Débogage — attention et vérification

> ⚠ Attention
>
> Corrigez **une** chose à la fois, puis relancez tous les tests : deux changements en même temps cachent lequel a réglé le problème.

- **Vérifiez** : dans l'exemple, à quelle ligne est l'erreur ?

Débogage — l4 et l5 (30 min)

1. Ensemble (5 min) : **l4-total-panier**, lisez l'erreur avant le code.
2. Seuls : corrigez l4, puis **l5-compter-jusqua** (un résultat faux).
3. Sur papier, pour chacun : l'erreur, la cause, la correction.
4. **Réussi si** : les deux au vert, et vos trois lignes écrites.

> 💡 Indice — l5
>
> Ajoutez `console.log(i)` dans la boucle et appelez `countTo(3)` : quelles valeurs de `i` passent ? Laquelle manque ?

Pour l4, ESLint trouve le problème avant même de lancer les tests. **Vous devez voir**, avant votre correction :

```
PS C:\Users\stagiaire\dwwm\semaine-03> npx eslint l4-total-panier.js

C:\Users\stagiaire\dwwm\semaine-03\l4-total-panier.js
  9:13  error  'totl' is not defined  no-undef

✖ 1 problem (1 error, 0 warnings)
```

`9:13` : ligne 9, colonne 13. `no-undef` : une variable utilisée sans avoir été déclarée. Après la correction, la même commande n'affiche plus rien.

Si le temps le permet — l6 (dernière lettre)

1. **l6-derniere-lettre** : deux problèmes, un résultat faux et une erreur.
2. Trouvez le test qui montre chacun des deux.
3. Corrigez-les un par un.

À retenir — ce matin

> À retenir
>
> - **Fonction** : `return` renvoie le résultat ; `console.log` affiche.
> - **Portée** : une variable n'existe que dans son bloc `{ … }`.
> - **Déboguer** : lire, isoler, tester, corriger ; une chose à la fois.

Le détail, notion par notion :

- **Fonction** : `function nom(paramètres) { … return …; }` ; déclarer n'exécute pas, appeler exécute ; gardez `export` dans les exercices.
- **Portée** : une variable `let` ou `const` n'existe que dans sa fonction ou son bloc ; hors de là, `ReferenceError`.
- **Déboguer** : lire le type, le message et la ligne de l'erreur ; isoler avec `console.log` ; tester le cas qui échoue ; corriger une chose à la fois, puis relancer tous les tests.

Débrief

- Quelle différence entre afficher un résultat et le renvoyer ?
- Pourquoi `result` n'existe-t-il pas en dehors de `double` ?
- Quelle étape de la démarche avez-vous le plus utilisée ?

Après-midi — TP — « Problem set 1 » (validé par les tests)

Apport 4 — Le reste `%` (10 min)

- **Pourquoi** : le dernier chiffre ; une division qui tombe juste.
- **`a % b`** : le reste de la division de `a` par `b`.

Pourquoi. Le Problem set de cet après-midi demande d'additionner les chiffres d'un nombre (et, si le temps le permet, de reconnaître un nombre premier). Il faut pour cela le dernier chiffre d'un nombre, et savoir si une division « tombe juste ».

Définition. L'opérateur reste `%` donne le reste de la division de `a` par `b` (MDN Web Docs, s. d.-h) : `1234 % 10` vaut `4`, le dernier chiffre ; `7 % 2` vaut `1`. Un nombre `n` est divisible par `d` si `n % d === 0`. Pour enlever le dernier chiffre : `Math.floor(1234 / 10)` vaut `123` (MDN Web Docs, s. d.-d). Le résultat de `%` a le signe du premier nombre : `-3 % 2` vaut `-1`.

Exemple.

```
console.log(1234 % 10);              // 4 : le dernier chiffre
console.log(Math.floor(1234 / 10)); // 123 : sans le dernier chiffre
console.log(12 % 3 === 0);           // true : 3 divise 12
```

Pas à pas. Le formateur montre chaque étape ; vous la refaites aussitôt.

1. REPL : `1234 % 10`, puis `123 % 10`, puis `12 % 10`.
2. `Math.floor(1234 / 10)` : le nombre sans son dernier chiffre.
3. `12 % 3` puis `12 % 5` : lequel « tombe juste » ?

Vous devez voir :

```
PS C:\Users\stagiaire\dwwm\semaine-03> node
Welcome to Node.js v24.21.0.
Type ".help" for more information.
> 1234 % 10
4
> 123 % 10
3
> 12 % 10
2
> Math.floor(1234 / 10)
123
> 12 % 3
0
> 12 % 5
2
```

Pour quitter le REPL : `.exit`, ou deux fois `Ctrl+C`.

Reste — attention et vérification

> ⚠ Attention
>
> Tester « impair » avec `n % 2 === 1` échoue sur les négatifs (`-3 % 2` vaut `-1`). Testez « pair » avec `n % 2 === 0`.

- **Vérifiez** : que vaut `57 % 10` ?

Consignes (1 h 50)

1. Quatre fonctions à écrire, dans l'ordre, fichiers `p1-…`.
2. Avant de coder : deux exemples d'entrée et de sortie, à la main.
3. Un test à la fois : lisez le test rouge, corrigez, relancez.
4. Fini quand ses tests et ESLint (sur son fichier) sont au vert.

Les quatre fonctions

- **p1-prix-ttc** (facile) : prix TTC arrondi au centime, `null` si négatif.
- **p1-bissextile** (facile) : année bissextile ou non.
- **p1-somme-chiffres** (moyen) : 1234 ⟶ 10, avec `%` et `while`.
- **p1-pgcd** (moyen) : l'algorithme d'Euclide.

Repères : la règle des années bissextiles du calendrier grégorien (Wikipédia, s. d.-b) ; l'algorithme d'Euclide (Wikipédia, s. d.-a) ; la définition d'un nombre premier (Wikipédia, s. d.-c). Pour arrondir : `Math.round` (MDN Web Docs, s. d.-e) ; pour enlever le dernier chiffre : `Math.floor` (MDN Web Docs, s. d.-d).

> Indice — p1-premier
>
> Essayez tous les diviseurs possibles à partir de 2 ; dès que l'un divise `n` (reste nul), `n` n'est pas premier. Pour aller plus vite, on peut s'arrêter quand `diviseur × diviseur` dépasse `n`.

Anglais technique — *Functions* (MDN)

Lisez la page *Functions* de MDN, en anglais (MDN Web Docs, s. d.-c).

1. Quels sont les titres des grandes sections ?
2. *Function declarations* : de quoi se compose une déclaration ?
3. *Function scopes* : une variable de la fonction est-elle visible dehors ?
4. D'après la première phrase, à quoi ressemble une fonction ?

Critères de réussite

- □ Les tests des quatre fonctions `p1-…` du noyau sont au vert
- □ ESLint ne signale rien sur ces fichiers
- □ Les quatre questions d'anglais ont une réponse

Si le temps le permet — p1-premier, puis p1-table

- **p1-premier** (moyen+) : nombre premier ou non, avec `%`.
- **p1-table** : la table de multiplication, sous forme de texte.
- Une ligne par multiplication ; lignes séparées par `\n`.

Pour aller plus loin

- Écrivez `daysInYear(year)` : 365 ou 366 jours.
- Réutilisez `isLeapYear` : une fonction peut en appeler une autre.

À retenir — cet après-midi

À retenir

- **Avant de coder** : deux exemples d'entrée et de sortie, à la main.
- `%` : le reste ; `n % d === 0` si `d` divise `n`.
- **Un test à la fois** : lire, corriger, relancer.

Sources

MDN Web Docs. s. d.-a. « export - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/export.

MDN Web Docs. s. d.-b. « Fonctions - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Functions.

MDN Web Docs. s. d.-c. « Functions - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions.

MDN Web Docs. s. d.-d. « Math : méthode statique floor() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Math/floor.

MDN Web Docs. s. d.-e. « Math : méthode statique round() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Math/round.

MDN Web Docs. s. d.-f. « Portée (Scope) - Glossaire ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Glossary/Scope.

MDN Web Docs. s. d.-g. « ReferenceError - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/ReferenceError.

MDN Web Docs. s. d.-h. « Reste (». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Operators/Remainder.

MDN Web Docs. s. d.-i. « return - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/return.

MDN Web Docs. s. d.-j. « TypeError - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/TypeError.

Wikipédia. s. d.-a. « Algorithme d'Euclide ». Consulté le 26 septembre 2026. https://fr.wikipedia.org/wiki/Algorithme_d%27Euclide.

Wikipédia. s. d.-b. « Année bissextile ». Consulté le 26 septembre 2026. https://fr.wikipedia.org/wiki/Ann%C3%A9e_bissextile.

Wikipédia. s. d.-c. « Nombre premier ». Consulté le 26 septembre 2026. https://fr.wikipedia.org/wiki/Nombre_premier.